

App Development-2 (MAD-2) Project Report

1. Student Details

- **Name** : Rahul Burdak
 - **Roll Number** : 23F2002371
 - **Email** : 23f2002371@ds.study.iitm.ac.in
 - **About Me** : I am a dual degree student pursuing Bachelor of Engineering in Information Technology from UIET, Chandigarh, Panjab University, along with a BS in Data Science and Applications from IIT Madras. I enjoy developing real-world applications using modern technologies and data-driven systems. I am always motivated to build solutions that solve practical problems and create meaningful impact.
-

2. Project Details

- **Project Title:**

Placement Portal Application

- **Problem Statement:**

Campus placement processes are often fragmented, involving multiple manual steps for students, companies, and administrators. Students struggle to track opportunities, companies face inefficiencies in managing applicants, and administrators lack centralized control and analytics.

The objective of this project is to develop a full-stack web application that streamlines the entire placement workflow enabling students to apply for drives, companies to manage recruitment, and administrators to oversee and control the process efficiently.

- **Approach:**

The application follows a **modular client-server architecture**. The backend is built using Flask and exposes RESTful APIs for handling authentication, role-based access, and data operations. The frontend is developed using Vue.js 3 as a Single Page Application (SPA) for a responsive and dynamic user experience.

To improve performance and scalability:

- **Redis** is used for caching frequently accessed data such as placement drives and dashboard statistics.
- **Celery** is integrated for handling asynchronous tasks like email notifications, CSV exports, and scheduled reports.

The system is designed with clear separation of concerns using Blueprints, reusable services, and optimized database queries.

3. AI/LLM Declaration

I used AI tools (ChatGPT) to assist in:

- Refactoring backend code for better structure
- Debugging API responses and query optimizations

- The extent of AI/LLM usage is around **15–17%**, mainly for debugging, restructuring, and improving code readability. Core logic, design decisions, and implementation were developed independently.

4. Technologies and Frameworks Used

Technology / Library Purpose

Flask	Backend framework for building REST APIs
Vue.js 3	Frontend framework for SPA
SQLAlchemy	ORM for database interactions
SQLite	Lightweight database for storing application data
Redis	Caching layer and message broker
Celery	Asynchronous task processing (emails, reports, exports)
Bootstrap 5	UI styling and responsive design
Flask-Mail / Mailhog	Email notifications and testing

5. Database Schema

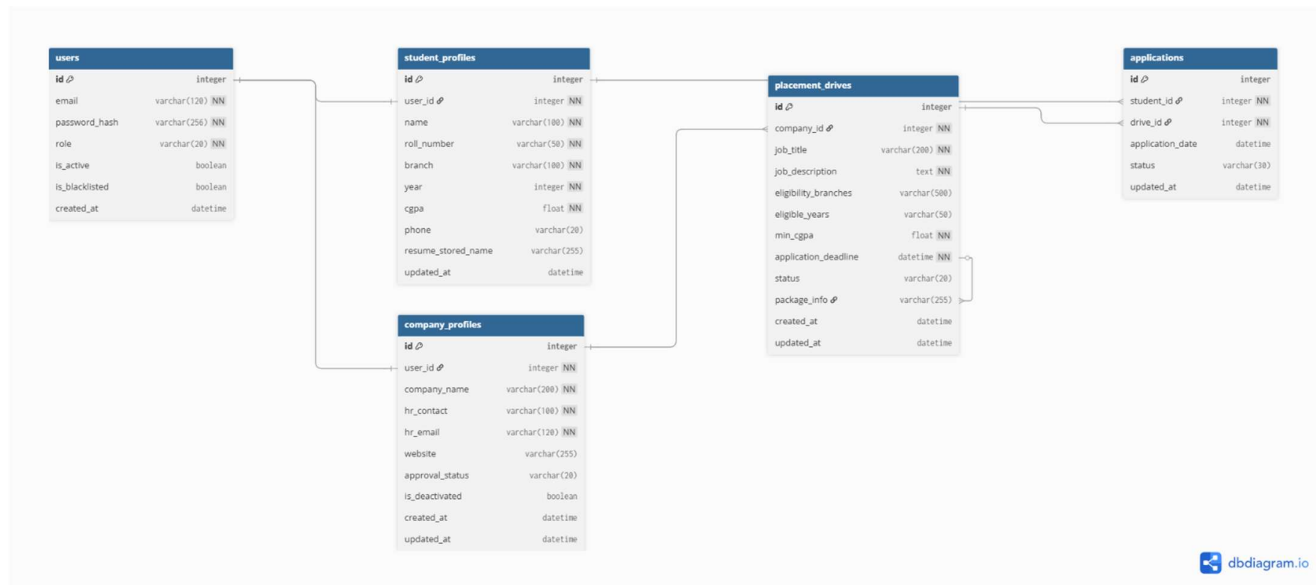


Figure 1: ER Diagram

Tables:

- **User**

Stores authentication and role information
(id, email, password_hash, role, is_active, is_blacklisted)

- **StudentProfile**

Stores student-specific data
(id, user_id, name, roll_number, branch, year, cgpa, phone)

- **CompanyProfile**

Stores company details

(id, user_id, company_name, hr_contact, hr_email, approval_status)

- **PlacementDrive**

Represents job opportunities

(id, company_id, job_title, description, eligibility, deadline, status)

- **Application**

Tracks student applications

(id, student_id, drive_id, status, application_date)

Relationships:

- One-to-One: User → StudentProfile
- One-to-One: User → CompanyProfile
- One-to-Many: CompanyProfile → PlacementDrive
- One-to-Many: PlacementDrive → Application
- One-to-Many: StudentProfile → Application

6. API Resource Endpoints

Endpoint	Method	Description
/auth/login	POST	Authenticate user and create session
/auth/register/student	POST	Register student account
/auth/register/company	POST	Register company account
/admin/dashboard	GET	Fetch system statistics
/admin/companies	GET	Manage company approvals
/admin/drives	GET	View and approve drives
/student/dashboard	GET	Fetch student dashboard data
/student/apply/{id}	POST	Apply to a placement drive
/company/drives	GET/POST	Manage company drives
/drives	GET	List available placement drives
/drives/{id}	GET	View drive details
/student/export-csv	POST	Trigger async CSV export

7. Architecture and Features

Architecture Overview:

- **Backend:**

- Flask app factory pattern for modular initialization
- Blueprints for separation of routes (auth, admin, student, company)
- SQLAlchemy ORM for database operations

- **Frontend:**

- Vue.js SPA with component-based architecture

- Role-based dashboards (Admin, Student, Company)
 - Bootstrap-based responsive UI
 - **Async Layer:**
 - Redis acts as broker and cache
 - Celery handles background jobs like email reminders and CSV exports
-

Implemented Features:

- **Role-Based Access Control:**
Separate dashboards and permissions for Admin, Student, and Company
 - **Placement Drive Management:**
Companies can create drives; admins approve them
 - **Application System:**
Students can apply to eligible drives with validation checks
 - **Eligibility Filtering:**
CGPA, branch, and year-based filtering
 - **Dashboard Analytics:**
Admin dashboard shows system statistics and insights
 - **Caching:**
Redis caching for frequently accessed endpoints (drives, dashboards)
-

Advanced Features (Background Jobs):

- **Async CSV Export:**
Students can export application history without blocking UI
- **Daily Email Reminders:**
Automated notifications for upcoming deadlines
- **Monthly Reports:**
Automated email reports summarizing placement activity

8. Video Presentation

Drive Link : [MAD2.mp4 - Google Drive](#)

Alternate Link : https://drive.google.com/file/d/1lmjkHC5mqAx_NG6aZvCy5iVVYX48hUjd